

A Window Cracked Open: Analyzing the Behavior of JavaScript Injected into In-App Browser Windows

Christian Jonas
Stanford University

Yavor Litchev
Stanford University

Michael Paper
Stanford University

Abstract

In-app web browsers are browsers directly embedded within mobile apps, where a link clicked on a mobile app will open a webpage and retrieve content into an in-app browser as opposed to opening a dedicated browser app (e.g. Chrome, Firefox, Safari, etc.) [1, 6]. Although seemingly benign, great amounts of control over said browser is given to the application itself. This can also pose a security risk. In particular, in-app web browsers can inject JavaScript (JS) into the pages they load for potentially nefarious reasons [1, 6]. We demonstrate in our work that these scripts can export every user action to untrustworthy entities, cause unpredictable actions to occur during the browsing experience, and any number of other attacks as a result of placing unvetted JS code at arbitrary parts of the page. As well as this, we conduct a study of the in-app browser activity of 31 apps. In particular, we extract the code that was injected within the browser in order to (1) perform similar analyses as prior works have done for other security vulnerabilities, (2) determine how grave of a threat these injections are, (3) determine how widespread in-app browser data exporting is, and (4) determine what could be done about this. Just as the authors of [7] argue that their analysis justify rethinking browser extensions from a security standpoint, we argue that a similar analysis of injected JS in in-app web browsers could lead to an interesting trade-off between programmability and security of the scripts allowed to be injected in in-app browsers.

1 Introduction

For most non-expert and some security-minded mobile app users alike, in-app browsers serve as a convenient feature of the app browsing experience, allowing them to quickly view web content in an app without having to navigate between apps. In-app browsers are pervasive, with numerous widely popular apps across app categories (e.g. Facebook, Instagram, Temu, etc.) utilizing them. For companies, they can serve as a subtle means of engagement maximization, keeping users

on an app. Additionally, due to the flexible structure of their implementation, they allow developers to tailor the in-app web-viewing experience to meet the custom needs of particular apps. This same flexibility lends itself to security pitfalls, which are the subject of our paper.

Recently, similar work has been done to examine browser extensions, which can interact with web pages with very few restrictions. Because popular browser extension stores (such as Google’s Chrome Web Store) have little enforced oversight, malicious extension developers can utilize JavaScript injection attacks to leak sensitive data. In [7], how much confidential user data they leak is measured, and a tool to automate this is presented.

Contributions Our methodology for examining this threat model consists of several high-level stages: (1) collecting apps on a mobile device, (2) collecting injected JS through visiting a web tool in each app’s in-app browser, (3) converting this JS to a browser extension, and (4) testing this JS for leaking sensitive information. We then use an application we made to prove the maliciousness of this attack. To summarize our work’s contributions, we present:

- A modest sample of iOS apps and the degree to which they collect and leak sensitive information (section 4).
- Evidence of the legitimacy of in-app browser JS injection as a viable attack vector (section 5).
- A demonstration of how pre-existing taint tracking tools (e.g. Arcanum and inappbrowser.com) can be adapted and orchestrated in order to examine novel information-leakage vulnerabilities (section 3).

The remainder of our paper has the following structure. First, in section 2, we discuss work that enabled our research and adjacent work in JavaScript-based information leakage attacks. In section 3, we discuss our methodology for collecting and analyzing our data. In section 4, we reveal our findings from this search. To bolster our claims, in section 5, we detail our creation of a custom app with a malicious in-app browser

to prove this attack vector’s legitimacy. Then, in [section 6](#), [section 7](#), and [section 8](#), we discuss our results and potential future work before concluding our work.

2 Related Work

Researchers have developed methods to trace how much information is leaked and tampered with via browser extensions. Such extensions typically have permissions to view and access all the content that the user is able to access, and as such they have enormous power to potentially run nefarious code. We believe that the mechanisms driving browser extension information leakage are similar enough to in-app browser information leakage for us to be able to adapt the methods of these papers.

In Xie et. al, researchers developed a dynamic “taint tracking” system called Arcanum [7], which allows researchers to tag certain portions of the webpage to identify “sinks” in the program where data can be maliciously exported. In particular, when the browser extension is activated (and potentially malicious code is injected), the V8 JS engine is leveraged to generate an Abstract Syntax Tree of the JavaScript, and the tagged portions can be logically traced to sinks within the program. Researchers were able to successfully demonstrate the tracing capabilities of this system. We leverage this tracing method, but use it to analyze in-app web browsers as opposed to extensions, which also have very high levels of access to the content displayed to the user.

Additionally, although novel methods of tracing and restricting information flow via leveraging Information Flow Control (IFC) (as opposed to Same Origin Policy (SOP)) [5] exist, the vast majority of sites and browsers ensure data isolation via SOP, which is outdated, leads to vulnerabilities, especially for embedded browsers. Additionally, beyond policy, there exist projects such as Hails that cleverly enforce mandatory access control and a declarative policy language to protect users on sites like Facebook and Yammer from third-party apps arbitrarily accessing information [2]. Still, the existence of robust systems does not mean that they are widely deployed, and it is still critical to create a tracing framework to identify what information is leaked via in-app web browsers.

Much of our work was inspired by the blog posts and tools developed by Felix Krause, who created [inappbrowser.com](#) [3]. In his blog post, he identifies the JavaScript file injected by Instagram and other apps owned by Meta, `pcm.js`. Details of `pcm.js` are given, and he identifies the potential for security and privacy violations being made through custom in-app browsers.

While we were writing our paper, on November 4th, a paper written by some of the authors of the Arcanum paper covering the same general domain as ours was published. We discuss this further in [subsection 6.2](#).

3 Methodology

In this section, we detail how we (1) gathered the scripts injected by apps’ in-app browsers and (2) analyzed these scripts through Arcanum. The approach is summarized by [Figure 2](#).

3.1 Getting the scripts

In order to conduct our study, we have to extract the scripts injected by apps. To do so, we manually ran apps that could open links, and tried to reach the page [inappbrowser.com](#) from all of them.

From some apps, such as Instagram, we just had to send a private message with a link to [inappbrowser.com](#) and click on that link. For other apps, where only a finite set of links are accessible, such as Google Maps, we had to find creative ways to get to [inappbrowser.com](#). One method that we have used repeatedly was to start signing in with Google’s OAUTH, but to access the privacy policy from the sign-in page. From Google’s privacy policy page, one can choose to be redirected to use any of Google’s services. Using Google search to search for [inappbrowser.com](#), we were finally able to open the webpage that would tell us if any script has been injected. [Figure 1](#) demonstrates our process for proceeding gathering a script from one app in our sample.

When a script is injected, [inappbrowser.com](#) detects it and shows the URL of the injected script.

3.2 Running an analysis on the script

Once the scripts have been extracted, we automatically turn them into browser extensions and run Arcanum [7] on them. Browser extensions consist mostly of injecting scripts into the user’s webpage, so this transformation is straightforward. To create the browser extension that emulates the injection of the in-app browser script `pcm.js`, we create a `manifest.json` file that looks like [Listing 1](#)

```
1 {
2   "name": "Facebook",
3   "permissions":
4     [
5       "activeTab",
6       "storage"
7     ],
8   "content_scripts":
9     [
10      {
11        "matches": ["<all_urls>"],
12        "all_frames": true,
13        "run_at": "document_end",
14        "js": ["pcm.js"]
15      }
16    ],
17   "manifest_version": 3
18 }
```

Listing 1: Manifest for `pcm.js` extension

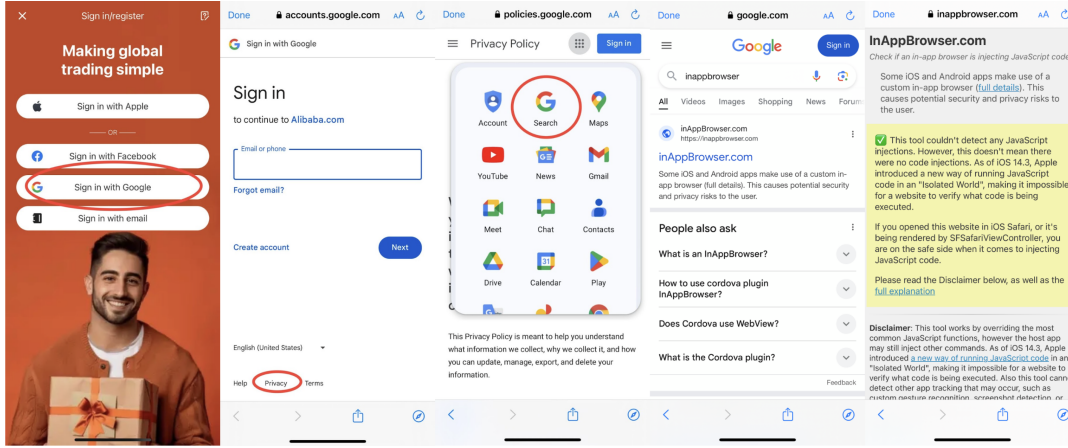


Figure 1: The progression from an app’s sign in page, to its in-app browser, and then to inappbrowser.com.

Arcanum then runs the security analysis of these extensions, which can be interpreted as the security analysis of the initial scripts.

4 Results

The list of apps that we inspected can be found in Figure 3. We chose these apps by looking at lists of popular apps and keeping those on which we thought we could reach a link to click on. We tried to test a diversified set of apps. We tested apps with developers from various countries, some open and some closed source apps.

The only apps that we have found to inject scripts are Instagram, Messenger and Snapchat. Instagram and Messenger both inject the same script.

We turned these two scripts into Chrome extensions. We ran Arcanum with these two extensions on the Facebook page that was manually annotated by the authors for taint tracking. Arcanum’s taint tracking found that none of the tainted data on that Facebook page got leaked into its sinks. In other words, no private data got leaked by injecting the scripts we extracted from Instagram, Snapchat and Messenger.

5 Attack Vector Validation

Through observation of the previous results, we were left unsure as to whether or not there was a problem with our detection mechanisms for JavaScript or if the sampled scripts were not in fact malicious, and therefore would not be detected by the combined usage of inappbrowser and Arcanum. To resolve this, we sought to demonstrate that there could potentially exist an in-app browser that would be detected via this system, and therefore our detection capabilities were not broken. However, as was previously stated, there was a fairly small number of applications that utilized a custom in-app browser system, and it would have been potentially infeasible

to continue searching for thousands of applications to test. Therefore, we decided that the correct direction was to construct our own app that would act in a malicious manner, and inject JavaScript into the contents of a webpage that would hopefully be detected by our system.

5.1 Custom Adversarial App Creation

To more precisely formulate our targets and constraints, we wished to create an application (Android or iOS would both have been possible, but Android was selected for feasibility reasons that will be explained later) such that when traversing through our procedure, it would produce a taint log of elements traveling from source to sink, and therefore indicate that information was leaked. It is of critical importance that the steps performed in the methodology are not modified, as we would like to show that our above methodology is capable of locating malicious JavaScript injections, and that our lack of producing a taint log is only a result of the detected JavaScript not being sufficiently malicious. Should we relax or tighten our procedure, it would fundamentally be a different process than what was used to produce the data procured in the results section, and therefore would not be useful in evaluating the merits of our process and the truthfulness of the results. Thus, we seek to create an application with an in-app browser such that when inappbrowser.com is visited, injected JavaScript is detected such that when said JavaScript is packed into a browser extension and processed by Arcanum, a valid taint log is created where information leakage is demonstrated.

In order to execute on this logic, we decided to create an Android application that would have a primitive "in-app browser," namely that there was an intended URL to be loaded that can be freely modified in code. Critically, irrespective of the URL, we wish to load adversarial JavaScript into the same web page that is separate from the intended URL. We

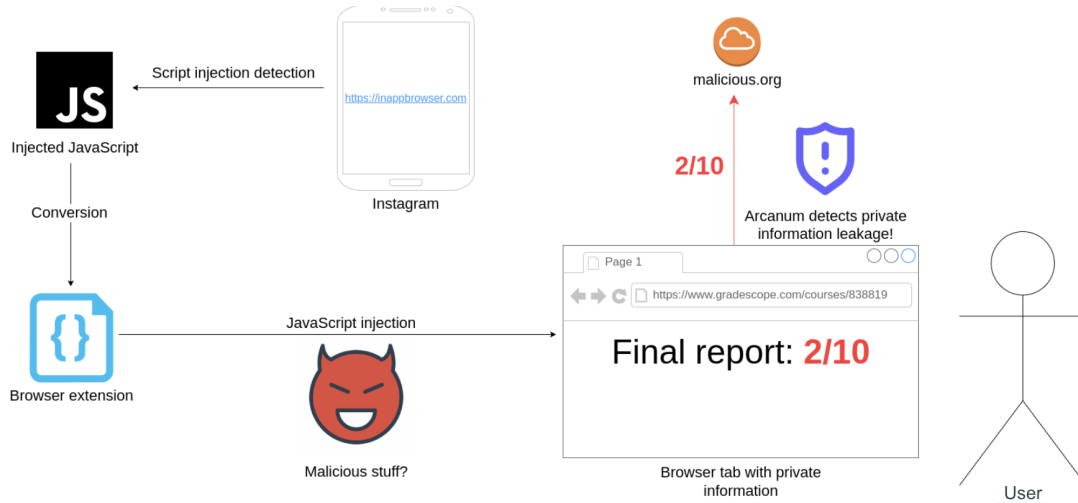


Figure 2: After detecting injected scripts, we convert them to browser extensions, and use Arcanum to detect if sources (e.g. variables storing `Final report: 2/10`) make it all the way to sinks (e.g. `malicious.org`)

would then set `inappbrowser.com` to be the intended URL to be loaded, and from there we would continue with our past procedures of packing the detected JavaScript into a browser extension and passing it into Arcanum. Android was selected due to its extensive libraries, documentation, open source nature, comprehensive SDK environment, and more, although this theoretically can also be accomplished with iOS. Additionally, we selected to use the WebView library to function as a primitive for retrieving web contents given an input URL, due to its extensive documentation, although other libraries for loading web contents could theoretically be used.

In our first attempt, we would perform a GET request load on a website that could host malicious JavaScript. Specifically, we chose the location of `pcm.js`: `https://connect.facebook.net/en_US/pcm.js`. Such scripts were stored in a string, which was then evaluated and executed in the web client prior to any other action occurring in the in-app browser. Finally, the browser loads the intended URL and displays it on the application page. It appeared that we were able to load some scripts, but they failed to be detected when the intended URL to be loaded was `inappbrowser.com`, as in `inappbrowser.com` did not display any indication that third party JavaScript was being loaded simultaneously with the current website load. This is in spite of the fact that we were clearly injecting Javascript separate from the intended URL to be loaded. Therefore, this approach was a failure.

In our second attempt, we more closely examined how `inappbrowser.com` detected JavaScript injections. As was discussed earlier, the website primarily leverages substring matching to find patterns associated with malicious scripts. One such pattern was `createElement('script')`, which could theoretically load any JavaScript element from the

web. We believe that this is how `pcm.js` was detected in the original blog post of the creator of the website `inappbrowser.com`. Therefore, for our new app, instead of requesting WebView to load JavaScript from a website and then insert it into the web contents of the requested URL, we created our own custom JavaScript that would call `createElement`, change the source of the script to be that of a website hosting malicious scripts, (such as the location of `pcm.js`), and inject this JavaScript prior to the load of the intended URL. More succinctly, instead of retrieving malicious JavaScript from a website via the application, we inject malicious JavaScript that retrieves malicious JavaScript. After some experimentation, `inappbrowser.com` was able to detect the JavaScript being loaded externally. Moreover, it correctly identified that the file loaded was `pcm.js`, and thus we had an application that not only injected scripts, but could be detected by the first part of our methodology.

5.2 Choice of injected JavaScript

To make the attack complete and fully successful, we substituted `pcm.js` with a JavaScript file that was present in one of Arcanum's test cases for creating a taint log with information being maliciously leaked. We hosted the JavaScript on a website that we own, and successfully were able to have `inappbrowser.com` detect said JavaScript when `inappbrowser.com` was the intended URL to be loaded. We then placed said JavaScript in a portion of a browser extension file, compiled it, and passed it to Arcanum. Arcanum was then able to successfully produce a taint log identical to that of the malicious extension we took the JavaScript from, and thus we succeeded in producing a malicious application that was detectable by our systems.

Category	App	Browser Behavior
Social	Instagram	Inject scripts
	Messenger	
	Snapchat	
	Twitter	Do not inject scripts
	Reddit	
	Discord	
	Reddit	
	LinkedIn	
	Slack	
	Google Maps	Open default browser
	TikTok	
	Mastodon	
	Whatsapp	
	Signal	
	Keybase	
	Element	
	Mattermost	
eCommerce	Temu	Do not inject scripts
	eBay	
	Uber	
	Wish	
	AliExpress	
	SHEIN	
	Alibaba	
	DHgate	
	Walmart	Open default browser
Misc.	New York Times	Do not inject scripts
	Washington Post	
	BetterHelp	Open default browser
	YouTube	
	Wikipedia	

Figure 3: List of apps we inspected, and the behavior they exhibited when we clicked on a link

5.3 Validation of methodology

As previously stated, we created an app that injects one of Arcanum’s own malicious test cases in its own in-app browser. Using this, we worked backwards to fetch the detected JavaScript through `inappbrowser.com`, and verified that Arcanum would produce a taint log for it. Through all of these steps, we demonstrate that our methods can indeed detect harmful JavaScript injections in in-app browsers. It is simply a requirement that they are sufficiently malicious so as to be detected by both `inappbrowser.com` and Arcanum. The methods through which this is accomplished might be artificial and contrived, as will be discussed in the following Discussion section, however this proves that our methods will not always report that all injected JavaScript from in-app browsers are perfectly safe. Thus, our methods can act as a distinguishing element between safe and harmful in-app browsers.

6 Discussion

We begin by noting that while JavaScript injection attacks are far from novel, the deployment of them within in-app browsers is a fairly new observation. While our results do not demonstrate that this is a saturated attack vector, while proceeding through our discussion, recall the demonstrated viability of this as an attack vector. With that said, in the wake of the Arcanum results, we discuss necessary facets that contextualize our results.

6.1 Comparison with Arcanum results

Given that we have demonstrated that malicious activity is feasible using in-app browsers, our results may initially seem surprising. Compared to [7], which found that a substantial fraction of browser extensions contain malicious data-exporting activity, we found little evidence of real apps’ in-app browsers doing the same. Because our work is closely tied with [7] and because their results seem to be the closest available academic results to ours, we seek to discuss this discrepancy. We have several key considerations that we believe drive this phenomenon.

Scope of In-App Browsers vs. Browser Extensions

Understanding the context of the usage of in-app browsers and the usage of browser extensions helps reveal why browser extensions are a more likely target for JS data leak attacks. In-app browsers typically are not intended for general-purpose web browsing, and most in-app browsers will never be used for sensitive purposes (e.g. banking). On the other hand, a browser extension is active across general purpose web browsing. This means that if it injects JS that leaks sensitive information, it will likely have an opportunity to do so in the normal course of a user’s expected browsing

behavior. This makes browser extensions a much more attractive target for malicious actors.

Ease of attack vector vs. existing attack vectors One key factor pertaining to this analysis is simply that it is more difficult to create an app and get it published on the iOS App Store than it is to create a browser extension and get it published on the Chrome Web Store; Apple’s review process requires that apps introduce some amount of novelty compared to existing apps, while the Chrome Web Store’s stated review process is far less holistic. Malicious actors are likely to have to do more work to create an app that gets approved compared to an extension, and in turn, would-be attackers are dissuaded from using an in-app browser based attack.

Store moderation and restrictions We only tested apps available on Apple iOS’s app store. Both the iOS App Store and Chrome Web Store provide guidelines on user data handling, but the vetting procedures for browser extensions is evidently insufficient. Our results indicate that even when code is injected into an in-app browser, it is not propagated outside of the app. One likely explanation that follows is that we don’t see this behavior from iOS apps because attempting to do so would cause the app to not pass Apple’s app review process, leading to us being unable to find malicious apps.

6.2 Concurrent work

During this quarter, some of the authors of Arcanum [7] published another paper [4] that is closely related to our work. This paper got published on November 4th, and we learned about it on December 5th.

[4] differs from our project in many ways. By decompiling apps, the authors found a way to automate the inspection of apps to find which ones have in-app browsers, which ones inject scripts into them, and what those scripts are. Therefore, the authors were able to run their analysis on thousands of apps retrieved from the Android Play Store. They only found 10 apps that inject scripts into their in-app browsers, so they analyzed those manually.

Their study did not find any app leaking confidential information. However, their most important find is that 3 apps (Moj, Chingari and Kik, which have respectively 289M, 97.5M and 176.5M downloads) injected ads into their browsers. Moj and Chingari are social media apps that are particularly popular in India and Kik is a messaging app.

It is worth noting that they did not identify any injected script for Snapchat for Android, but we did detect it on Snapchat for iOS with `inappbrowser.com`.

6.3 Limitations

Firstly, we were limited in our set of testable apps to locate in-app browser extensions. We only had access to mobile devices running iOS for app testing purposes. As such, we note that our results may not generalize to mobile app stores with fewer restrictions than Apple’s App Store. We suggest as a direction for future work that the methodologies developed in our work be replicated by those with access to alternative app stores, such as the Google Play store.

Additionally, while the attack vector validation section proves that our method is capable of detecting injected JavaScript, it was a fairly contrived and specific example that. As was stated in the earlier section, we had to change our injection method between attempts in order for `inappbrowser.com` to detect external scripts being loaded, which raises the question as to the reliability of `inappbrowser.com` detecting arbitrary JavaScript injections. It is quite possible that this method did not properly signal when an in-app browser was injecting JavaScript, and therefore some of the applications marked as safe could be malicious.

7 Future Research

Although substantial progress has been made, there are many domains in which additional progress can be made in order to make such detections more effective. For example, in order to get around the limitation of reduced quantities of JavaScript that was retrieved by `inappbrowser.com`, one potential future research direction is to leverage an emulator to analyze memory of the Android apps, and retrieve JavaScript via inspection of the RAM. We partially attempted this, and surprisingly we were able to retrieve some sparse quantities of JavaScript, by virtue of the fact that the JavaScript would most likely be stored in plaintext on the system’s memory (as the OS would control access to its memory, encrypting its own information seems unlikely given that we did not specify the emulation of a trusted execution environment). However, we believe that with further analysis, greater quantities of JavaScript could be retrieved, and we could thus provide Arcanum with greater amounts of information to analyze.

Another potential research direction in order to retrieve greater quantities of JavaScript is to fork `inappbrowser.com` and relax detection techniques. Fundamentally, it appears that `inappbrowser.com` leverages string matching techniques to identify substrings/patterns of JavaScript that can be classified as suspicious, and report such instances. However, a clever attacker can leverage many techniques to avoid the detection algorithms present, and even without a clever attacker, there are malicious patterns of JavaScript that can be executed without being detected by `inappbrowser.com`. For example, in our mock Android application, injecting the scripts within `pcm.js` directly into the browser was not detected, but a script

that loaded `pcm.js` via a call to `createElement('script')` was successfully detected. Thus, modifying the source code of `inappbrowser.com` could allow for greater quantities of retrieved JavaScript.

Finally, although the existing classification of sources and sinks are thorough and fairly exhaustive in Arcanum, we believe that a future project could examine more closely which sources and sinks are more prevalent in in-app browsers. Ultimately, both in-app browsers and browser extensions have complete access to the JavaScript and webpage being displayed, and therefore many of the methods for leaking information would be similar between the two attack vectors. Nonetheless, relaxing the constraints of information flow from source to sink via adding more sources and sinks can allow for a broader analysis of different attacks, and therefore allow for a more comprehensive taint log to be produced.

8 Conclusion

To conclude, we have created a detection scheme for JavaScript injections performed on in-app browsers, and evaluated the safety and integrity of said injections on a diverse range of apps. We also created our own custom application on Android to act as a malicious in-app browser, and demonstrated that our detection systems can indeed identify a malicious application with an in-app browser that injects JavaScript. We build off of prior work examining JavaScript injections using browser extensions, and we have thus contributed to the body of work examining malicious injections and tracing the extent of how such injections may be damaging.

This project contributes to the domain of detecting adversarial and malicious attacks, but ultimately it is in the interest of everyone to prevent such attacks in the first place. Fundamentally, these attacks are possible because applications in the Android and iOS store are allowed to create their own methods of parsing and executing JavaScript. One solution would be for Android and iOS to prevent loads from arbitrary Application elements, and instead require that applications interact with an API for loading external links. Such an API can ultimately leverage Chrome, Firefox, or some other open source. Although creating such an API is likely not too challenging (and probably already exists), enforcing applications to only load external links via said API could be quite challenging, as one would have to detect that applications do not implement their own JavaScript or web execution engines. Machine learning could be leveraged to analyze existing compilers and attempt to identify applications that wish to load and execute custom scripts from the web via their own compilers, however such a process is likely to be burdensome, challenging, and niche. Nonetheless, until a prevention mechanism is created, it is crucial that detection mechanisms are created and deployed, so that the public is at least aware that their information is not as secure as they might believe.

References

- [1] ESCH-LAURENT, P. The pitfalls of in-app browsers, 2024.
- [2] GIFFIN, D. B., LEVY, A., STEFAN, D., TEREI, D., MAZIÈRES, D., MITCHELL, J. C., AND RUSSO, A. Hails: Protecting data privacy in untrusted web applications. In *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12)* (Hollywood, CA, Oct. 2012), USENIX Association, pp. 47–60.
- [3] KRAUSE, F. ios privacy: Instagram and facebook can track anything you do on any website in their in-app browser, 2022.
- [4] KUCHHAL, D., RAMAKRISHNAN, K., AND LI, F. Whatcha lookin’ at: Investigating third-party web content in popular android apps. In *Proceedings of the 2024 ACM on Internet Measurement Conference* (New York, NY, USA, 2024), IMC ’24, Association for Computing Machinery, p. 114–129.
- [5] STEFAN, D., YANG, E. Z., MARCHENKO, P., RUSSO, A., HERMAN, D., KARP, B., AND MAZIÈRES, D. Protecting users by confining JavaScript with COWL. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)* (Broomfield, CO, Oct. 2014), USENIX Association, pp. 131–146.
- [6] STEINER, T. Inspecting facebook’s webview, 2019.
- [7] XIE, Q., MURALI, M. V. K., PEARCE, P., AND LI, F. Arcanum: Detecting and evaluating the privacy risks of browser extensions on web pages and web content. In *33rd USENIX Security Symposium (USENIX Security 24)* (Philadelphia, PA, Aug. 2024), USENIX Association, pp. 4607–4624.